

RG 全体講義: プロトコル設計における脅威の明確化と正当性証明

moz@sfc.wide.ad.jp

April 30, 2025

主題

- ・ プロトコルの設計について紹介する
- ・ 脅威モデルについて紹介する
- ・ プロトコルの正当性証明 (Formal Proof) について紹介する
- ・ 最後に Delight 的な (分散システムとデジタルアイデンティティについての) 研究課題を紹介する

目的

- ・ 分散システムにおけるプロトコルの設計 (のお気持ち) を理解する
- ・ プロトコルの設計における正当性保証の考え方 (のお気持ち) を理解する
- ・ 自身のプロトコルを学術論文としてまとめる際、どのような構成で何を明らかにすれば良いかを理解する

プロトコルとは？

プロトコルの定義 (Oxford Dictionary)

A set of rules governing the exchange or transmission of data between entities.

エンティティ：プロトコルに参加する個々の主体（ユーザー、デバイス）を指す。

アルゴリズムの定義 (Oxford Dictionary)

A set of instructions for performing a task or solving a problem.

アルゴリズムは計算手順に焦点を当てるのに対し、プロトコルは参加者間の相互作用に重点を置く。¹一般的なプロトコルの例：HTTP、SMTP、TLS/SSL、SSH、OAuth、Paxos、Raft

¹プロトコルは英語や日本語など共通のルールで会話する例として説明されることが多いが、これは相互作用の規則を表しているためである。一方で「日本語アルゴリズム」という表現が一般的でないのは、アルゴリズムが相互作用ではなく計算手順を表すためである。

この講義はプロトコルの設計手法については紹介しない。

皆さんが自分のドメインで設計したプロトコルがなぜ正当 (Correctness) と言えるのかをどのように他人に説明できるかを考える講義

プロトコルの正当性を示すには、以下の点を明確にすることが重要である：

- ・ どのようなエンティティがプロトコルに関与するか
- ・ エンティティ間でどのような情報交換が行われるか
- ・ プロトコルはどのような条件を満たせば安全であると言えるのか
- ・ それぞれのエンティティがどのような振る舞いをする可能性があるか
- ・ そのようなエンティティが存在するときに、プロトコルが安全であるかどうかをどのように判断するか

敵対者と脅威モデル

プロトコル設計において、どのような振る舞いを許容するかを考える上では、エンティティの能力と行動に関する仮定を明確にすることが重要である。

敵対者

An adversary refers to an attacker, typically with malicious intent, who undertakes actions against a system or protocol

敵対者は、プロトコルに参加するエンティティを制御下に置き、そのエンティティの動作を変更または監視できる状態。

脅威モデル

脅威モデル (threat model) とは、敵対者の能力と行動に関する仮定を形式化したモデル。



脅威モデルの構成要素

脅威モデルは以下の5つの要素から構成から構成される：

1. Corruption Model. 敵対者が制御するエンティティの行動様式
2. Computational Power. 敵対者が利用できる計算リソースの制約
3. Adaptivity. 敵対者がエンティティを腐敗させるタイミングと方法
4. Visibility. 敵対者が観察できる情報の範囲
5. Mobility. 敵対者によるエンティティの腐敗と回復のパターン

この要素の組み合わせ方によって、異なる脅威モデルが定義される。どのような要素を選択するかは、プロトコルの目的、実行されうる環境によって決まる。

腐敗モデル (Corruption Models)

腐敗モデルは、敵対者はエンティティをどのように制御できるかを定義する。

- ・ Passive. プロトコルに完全に従いながら、観測したすべての情報を蓄積して活用できる。しかし、プロトコルの仕様に逸脱することはできない。²³
- ・ Crash. 応答を任意のタイミングで停止できるが、停止時間外は正確にプロトコルに従って動作する。
- ・ Omission. エンティティは受信したのメッセージを任意に削除または無視できる。送信するメッセージの内容を変更することはできない。
- ・ Byzantine. エンティティはプロトコル仕様から任意に逸脱できる。

各モデルは前のモデルを包含している：Byzantine ⊃ Omission ⊃ Crash ⊃ Passive

²具体例として銀行の従業員、顧客の取引データを正しく処理しながらも、そのデータをこっそり記録して個人的に利用する場合。

³Honest-but-Curious, Semi-Honest モデルとも呼ばれる。

適応性モデル (Adaptivity Models)

適応性モデルは、敵対者がいつ、どのように参加者を腐敗させるかを定義する：

- ・ Static：敵対者は腐敗させるエンティティをプロトコル実行前に選ぶ。実行中に変更できない。
- ・ **Adaptive**：敵対者はプロトコル実行中に腐敗させるエンティティを動的に選択できる。

エンティティが少ない場合、プロトコルの通信フェーズ数が多い場合、プロトコルの実行時間が長い場合、Adaptive モデルを考慮する。例えば、鍵交換モデルなどでは、Static モデルを考えることが一般的。コンセンサスアルゴリズムでは、Adaptive モデルを考えることが一般的。

可視性モデル (Visibility Models)

可視性モデルは、敵対者がどの程度のメッセージや内部状態を観察できるかを定義する。

- ・ Full Information. 敵対者はすべてのメッセージ内容と各エンティティの内部状態（実行中の変数）を完全に観察可能
- ・ Private Channels. 正直なエンティティ間の通信内容は敵対者から隠されるが、通信の発生事実やメタデータ（送信元、宛先、タイミング、メッセージサイズ等）は観察可能。腐敗したエンティティの状態、メッセージ内容は観察可能。
- ・ Erasure Model. Private Channels に対して、参加者が腐敗する前に消去されたデータは、敵対者は観察・復元できない制約を足したもの。
- ・ Oblivious. メッセージのヘッダー情報（送信元、宛先、長さ）のみを観察可能で、内容は完全に隠蔽される。⁴

暗号プロトコルでは、機密性 (intended to be kept secret) を保証するために、Private Channels モデルを考えることが多い。

⁴Trusted Computing のようなシステムでは、このモデルを考えることが多い。

計算能力モデル (Computational Power Models)

計算能力モデルは、敵対者の計算資源の制限を定義する：

- ・ Unbounded. 無制限の計算資源を持つ敵対者。理論上はすべての暗号を破ることができる。
- ・ Polynomially Bounded. 多項式時間の計算⁵に制限される敵対者
- ・ Fine-grained. 特定の計算制限を持つ敵対者。
秒間ハッシュ計算回数に制限があるモデル (Hashrate)

重要なのは、エンティティの計算能力ではなく、敵対者の計算能力である。敵対者は Visibility Models にそって、エンティティのメッセージや内部状態を観察できる。その情報を敵対者はどのような計算資源を用いて活用できるかを定義する。

⁵ある問題 (Problem) を解くときに、その問題の解を求める計算複雑性がオーダー $O(n^c)$ $n, c \in \mathbb{Z}$ で表せる。
例えば、離散対数問題、素因数分解問題の難しさなど。

一般的には Probabilistic Polynomial Time (PPT) が確率チューリングマシンにおいて、入力となるセキュリティパラメータ $\lambda \in \mathbb{Z}$ (鍵長など) の大きさに対して、乱数を用いながらも 多項式時間 λ^c 以内に必ず終了するアルゴリズム

移動性モデル (Mobility Models)

移動性モデルは、敵対者による参加者の腐敗と回復のパターンを定義する：

- ・ Fixed. 一度腐敗したエンティティはプロトコル実行中ずっと腐敗したままであり、回復することはない
- ・ Mobile. 敵対者はエンティティを動的に腐敗させたり、回復させたりできる。ただし、任意の時点で腐敗しているエンティティの総数には上限がある⁶

IoT やモバイルデバイス、システムの長期運用の視点を考えると、一般的に Mobile モデルを考える。

⁶Adaptivity Models の Adaptive との混同に注意すること。Adaptive は敵対者がエンティティを腐敗させるタイミングを選択できることを指す。Mobility はその条件下において、一度腐敗させたエンティティが正常なエンティティとして回復するかどうかを考慮している。

適切な仮定を置くことの重要性

セキュリティプロトコルの研究において、**仮定の明確化**は極めて重要である。

- ・ 仮定が弱すぎる場合. 現実的な攻撃に対して脆弱なプロトコルになる可能性がある
- ・ 仮定が強すぎる場合. 設計や実装が複雑になり、その影響でパフォーマンスが下がり、実用性が低下する可能性がある

仮定を明示する価値：

- ・ **プロトコルの正当性**を正確に理解できる
- ・ 異なるプロトコルを同じ基準で公正に比較できる

実際のシステム設計では、特定の用途や発生しうるリスクに基づいて適切な仮定（脅威モデル）を選択することが重要である。

例：Practical Byzantine Fault Tolerance (PBFT)

PBFT

- ・ PBFT は、Byzantine 障害を許容しつつも、Linearizable⁷を達成するレプリケーションプロトコル⁸である。
- ・ ただし、 $n \geq 3f + 1$ （ここで f は敵対者の数、 n はレプリカの総数）のときに限る。

PBFT プロトコルは、非同期通信モデルにおいて初めて Linearizable を達成することを示したプロトコルである。発表から 20 年経った現在でも、PBFT は依然として広く使われている。

⁷Linearizable に複製するとは、すべてのレプリカが同じ順序でクライアント要求を処理し、その反映がクライアントの Read リクエストのセッション内で必ず行われる性質である。各複製データにはシーケンス番号が割り当てられ、このシーケンス番号に従って処理される。

⁸チューリング完全な状態マシンでは、複製されるデータをオペレーション（状態遷移関数）と見なす。オペレーション f, g がある場合、一般に $f \circ g \neq g \circ f \neq id$ であるため、実行順序が重要になる。

PBFT のプロトコル詳細 (1)

PBFT の主なフェーズ：

1. Pre-prepare：プライマリ（リーダー） p がクライアント要求 r にシーケンス番号 s を割り当て、 $\langle \text{PRE-PREPARE}, s, r \rangle$ メッセージを他のレプリカに送信
2. Prepare：レプリカが要求の順序に合意し、 $\langle \text{PREPARE}, s, r \rangle$ メッセージを全レプリカに送信
3. Commit：少なくとも $2f + 1$ の PREPARE メッセージを受信したレプリカは $\langle \text{COMMIT}, s, r \rangle$ メッセージを送信し、実行を確約
4. Reply： $2f + 1$ の COMMIT メッセージを受信したレプリカは要求を実行し、結果をクライアントに返送



PBFT の脅威モデル

PBFT の脅威モデルを考えてみよう。

PBFT の脅威モデル Λ_{PBFT}

- Corruption : **Byzantine** - レプリカは異なるメッセージを異なるレプリカに送信するなど、任意に振る舞うことができる
- Adaptivity : Adaptive - 敵対者はプロトコルの各フェーズで異なるレプリカを動的に腐敗させられる
- Computational Power : Polynomially Bounded - 多項式時間の計算に制限される（暗号署名が偽造不可能と仮定）
- Visibility : Private channels - 正直なレプリカ間の通信内容は秘密であり、内部状態も敵対者からは見えない
- Mobility : Mobile - 敵対者はプロトコルの各フェーズで異なるレプリカを動的に腐敗させられる

PBFT の正当性保証

$n \geq 3f + 1$ のとき (f は腐敗ノードの数、 n はレプリカの総数) :

- ・ **安全性** (Safety) : 正直なレプリカが同じシーケンス番号で異なる値をコミットすることはない
- ・ **活性** (Liveness) : すべてのクライアント要求は最終的に処理され応答が返される

を満たすことを証明できれば、PBFT は正当 (Correct) であると言える。

すなわち敵対者が脅威モデル Λ_{PBFT} の範囲においてどのような攻撃を行っても、Safety と Liveness を満たすことを証明できれば、PBFT は正当 (Correct) であると言える。

Question: なぜ $n \geq 3f + 1$ のとき Safety が満たされるのか？ PBFT がセキュアであることを説明してみよう！

仮定: 二つの異なる値 v と v' が同じシーケンス番号 s でコミットされたと仮定する。

Lemma

各値がコミットされるためには、少なくとも $2f + 1$ 個のレプリカから *PREPARE* メッセージが必要である。 $(3f + 1$ 個のレプリカのうち、腐敗したレプリカは最大 f 個)

- ・ 値 v がコミットされるには少なくとも $2f + 1$ 個のレプリカ $\mathcal{P}$ からの *PREPARE* メッセージが必要
- ・ 同様に、値 v' がコミットされるには少なくとも $2f + 1$ 個のレプリカ $\mathcal{Q}$ からの *PREPARE* メッセージが必要
- ・ 合計 $n \leq 3f + 1$ レプリカのうち、悪意あるレプリカは最大 f 個
- ・ $|\mathcal{P} \cap \mathcal{Q}| \geq 2f + 1 + 2f + 1 - n \geq 4f + 2 - (3f + 1) = f + 1$
- ・ つまり、少なくとも $f + 1$ 個の**正直なレプリカ**が両方の値 v と v' について *PREPARE* メッセージを送信したことになる
- ・ しかし、許容する腐敗レプリカは最大で f 個であって、正直なレプリカは同じシーケンス番号に対して複数の値に *PREPARE* を送信しない（プロトコルの定義により）
- ・ これは矛盾であり、したがって仮定は誤り

合成可能性問題 (Composability Problem)

ちょっとお題が変わって、複雑なプロトコルの証明について考えてみよう。

分散システムでは、既存のプロトコル上に新しいプロトコルを構築することが一般的である⁹

Motivation：複数のプロトコル $\pi_1, \pi_2, \dots, \pi_n$ があったとき、それらを組み合わせたプロトコル $\Pi^{\pi_1, \pi_2, \dots, \pi_n}$ の証明はどう行うか？

システム全体を最初から再証明するのは非常に複雑で時間がかかる。

⁹例えば、PBFT を使用する Hyperledger Fabric 上に新たなアプリケーションを構築

解決策の一つ：Universal Composability (UC) を用いたプロトコルの安全性証明セキュリティ分析において考慮される攻撃の範囲と限界を明示的に定義する。明示的な脅威モデリングが不可欠である。

UC は、プロトコルを部品の集合として捉え、これらの部品を合成したプロトコルのセキュリティを保証する。これにより、複雑なシステムの分析を、より単純な構成要素の分析に還元できる。

Universal Composability Framework

Universal Composability (UC) の中心的なアイデアは、**理想/現実世界パラダイム**である：

理想世界 (Ideal World)：

- ・ 理想機能 $\mathcal{F}$ が直接タスクを安全に実行する
- ・ これは「定義上安全」な仕様を表す抽象的なエンティティ
- ・ 例：理想的な署名機能 $\mathcal{F}_{\text{sig}}$ は、正当な署名者のみが署名を生成でき、偽造が不可能であることを保証

現実世界 (Real World)：

- ・ 実際のプロトコル Π が参加者によって実装され、敵対者 $\mathcal{A}$ の攻撃に晒される
- ・ 敵対者は、前述の脅威モデル（腐敗、適応性、計算能力等）に基づいて定義される
- ・ 攻撃に晒されるということは、理想世界のような「定義上の安全性」が自明でなく、証明が必要になる

Universal Composability Framework (続き)

環境 $\mathcal{Z}$:

- ・ プロトコル外部のすべてのシステム (ユーザー、他のプロトコル等) を表現する
- ・ 入力を提供し (例: 署名すべきメッセージ)、出力を受け取る (例: 署名された結果)
- ・ 環境はプロトコル参加者と敵対者の両方と相互作用し、どちらの世界 (理想/現実) で動作しているかを区別しようとする

シミュレータ $\mathcal{S}$:

- ・ 理想世界で敵対者を模倣するエンティティ
- ・ 現実世界の敵対者の振る舞いを理想世界でシミュレートし、環境から見て区別不可能にする

セキュリティの定義

任意の敵対者 $\mathcal{A}$ に対して、シミュレータ $\mathcal{S}$ が存在し、どんな環境 $\mathcal{Z}$ も以下を区別できないこと:

- ・ 実際のプロトコル Π と敵対者 $\mathcal{A}$ との対話

本当に仮定はこれだけでいいのか？

ここまでで用いた仮定は以下の通りである：

- ・ ネットワークモデル：非同期か同期か、部分同期（通信遅延に上限がある）か
- ・ 敵対者モデル：敵対者はどのように振る舞うか
- ・ 腐敗ノードの数：全体の何割まで敵対者が制御できるか ($n \geq 3f + 1$ など)

Question: 実世界で動かす上で考えるべき脅威、仮定はこれだけか？

これだけではすべてのプロトコルに Sybil Attack が適用できてしまう。

Sybil Attack

単一の実体が複数の偽の身元を作成し、システム内で不当な影響力を得る攻撃である。

例：PBFT では $n \geq 3f + 1$ の条件がある。 n はノードの数、 f は敵対者の数である。敵対者が単一のエンティティ（敵対者）が複数のエンティティとして振る舞えることを許すと、容易に条件を破ることができる。

アイデンティティの生成コスト

公開鍵を識別子として用いる場合、大抵の計算機では公開鍵は計算コストが低いため、Sybil Attack に脆弱である。

単にノードが公開鍵に対応する秘密鍵を持っていることをデジタル署名を用いて示すだけでは、そのノードが独立した実体¹⁰であることを保証できない。

単一のエンティティと公開鍵の一対一対応を保証するためには、認証局による一対一対応の検証と証明が必要である。

- ・ 認証局の存在を許す = 信頼できる第三者の存在を仮定する
- ・ 単一障害点とスケーラビリティの問題が生じる¹¹

ほかにも Proof-of-Work/Stake：新たな識別子の作成に計算資源や経済的資源を要求する研究もある。

¹⁰ エンティティをどのように区別し、何を持って独立と定義するのかは議論の余地がある。

¹¹ あるエンティティが自分は山田花子と主張したが、本当に山田花子であるかどうかを検証したあとに、山田花子とそのエンティティが提示した公開鍵を紐付ける。その紐付きに対して認証局が署名を行い、証明書を作成する。